

Java'da Variables, Data Types ve Operators

Temel yapı taşlarından Control Statements'a giriş

Java Programlama · Dr. Öğr. Üyesi Baki Gökgöz · Gümüşhane Üniversitesi

Bu Derste Neler Öğreneceğiz?

Variables

Variable nedir, nasıl tanımlanır? Local, instance ve static variable farkları; isimlendirme kuralları.

Data Types

8 primitive type, non-primitive (reference) type'lar, wrapper class'lar ve type casting.

Operators

Arithmetic, unary, relational, logical, bitwise, shift, assignment ve ternary operator'ler; precedence.

Control Statements

Program akışını değiştiren decision-making, loop ve jump statement'lara genel bakış.

Bölüm 1



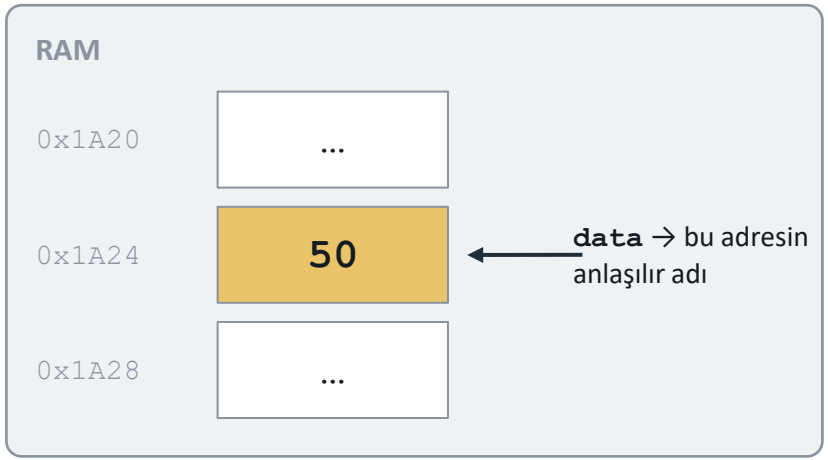
Variables

Bellekteki verilere isim vermek

Variable Nedir?

- Variable, bellekte (RAM) ayrılmış bir alana verilen **isimdir**.
- Adı "vary + able" kelimelerinden gelir: program çalışırken değeri **değişebilir**.
- Java **statically typed** bir dildir: her variable'ın bir **data type**'ı vardır ve compile time'da bilinir.
- Data type; variable'ın **ne tür** veri tutacağını ve bellekte **ne kadar yer** kaplayacağını belirler.





Declaration, Initialization ve Assignment

```java
int age;  // declaration
age = 21;  // assignment
int year = 2026;  // declaration + initialization
int a = 5, b = 10;  // tek satırda birden fazla
final double PI = 3.14159;  // constant (sabit)
var city = "Gümüşhane";  // Java 10+: type inference

age = age + 1;  // değer değişebilir
System.out.println(age + " " + year);
System.out.println(a + b);
System.out.println(PI + " " + city);
// PI = 3.14;  -> compile error: final variable
```

Çıktı (Output)

```
22 2026
15
3.14159 Gümüşhane
```

- **Declaration**: tip + isim; bellekte yer ayrılır.
- **Initialization**: variable'a ilk değerin verilmesi.
- **`final`** variable yalnızca bir kez atanır; sabitler genelde UPPER_CASE yazılır.
- **`var`** tipi sağ taraftan çıkarır; Java yine statically typed'dır (**`var`** sonradan başka tip alamaz).

Identifier (İsimlendirme) Kuralları

Kurallar (zorunlu)

- Harf, rakam, `_` ve `$` içerebilir
- Rakamla **başlayamaz**
- Boşluk ve `-` içeremez
- Reserved keyword olamaz (`int`, `class`, `for`…)
- **Case-sensitive**: `age` ≠ `Age`

Konvansiyonlar (önerilen)

- variable / method: **camelCase** → `studentName`
- class: **PascalCase** → `Student`
- constant: **UPPER_SNAKE_CASE** → `MAX_SIZE`
- Anlamlı isim: `x` yerine `totalPrice`

Örnekler

- Geçerli: `count`, `_temp`, `$price`, `student2`
- Geçersiz: `2student` (rakamla başlıyor)
- Geçersiz: `my-var` (tire)
- Geçersiz: `class` (keyword)
- Geçersiz: `first name` (boşluk)

Java'da 3 Tür Variable



Local Variable

- Method, constructor veya block `{ }` içinde tanımlanır
- Yalnızca kendi **scope**'unda görünür
- **Default değeri yoktur**: kullanmadan önce initialize edilmelidir
- `static` olamaz



Instance Variable

- Class içinde, method dışında, `static` olmadan tanımlanır
- Her object'in **kendi kopyası** vardır
- `new` ile object oluşturulunca yaratılır
- Default değer alır: `0`, `false`, `null`



Static Variable

- Class içinde `static` keyword ile tanımlanır
- Tüm object'ler **tek bir kopyayı paylaşır**
- Class belleğe yüklenince bir kez oluşur
- `ClassName.variable` ile erişim önerilir

Local Variable ve Scope

```java
public class Main {
    public static void main(String[] args) {
        int num = 10;              // local variable
        System.out.println("num: " + num);

        for (int i = 0; i < 2; i++) {
            int square = i * i;    // sadece bu block'ta
            System.out.println("square: " + square);
        }
        // System.out.println(square); // HATA: scope dışı

        int total;                 // henüz değer yok
        // System.out.println(total);  // HATA!
        total = 5;
        System.out.println("total: " + total);
    }
}
```

Çıktı (Output)

```
num: 10
square: 0
square: 1
total: 5
```

- **Scope**: variable'ın görünür olduğu alan; tanımlandığı `{ }` block'u ile sınırlıdır.
- `square` ve `i`, for block'u bitince yok olur.
- Initialize edilmemiş local variable → **"might not have been initialized"** hatası.

Instance Variable: Her Object'in Kendi Kopyası

```java
class Student {
    String name;          // instance variable
    int age = 19;         // instance variable

    Student(String name) {    // constructor
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Ali");
        Student s2 = new Student("Ayşe");
        s2.age = 21;
        System.out.println(s1.name + " - " + s1.age);
        System.out.println(s2.name + " - " + s2.age);
    }
}
```

Çıktı (Output)

```
Ali - 19
Ayşe - 21
```

- **`s1`** ve **`s2`** ayrı object'lerdir; her birinin **`name`** ve **`age`** değeri ayrıdır.
- **`s2.age`** değişince **`s1.age`** **etkilenmez**.
- **`this.name`** object'in field'ını, **`name`** ise constructor parametresini gösterir.
- Bir **`.java`** dosyasında yalnızca **bir** **`public class`** olabilir.

Static Variable: Tüm Object'lerin Ortak Kopyası

```java
class Student {
    static int count = 0;    // static: paylaşılır
    String name;             // instance

    Student(String name) {
        this.name = name;
        count++;             // her yeni object'te artar
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Ali");
        Student s2 = new Student("Ayşe");
        Student s3 = new Student("Mehmet");
        System.out.println("Toplam: " + Student.count);
        s1.count = 99;       // önerilmez!
        System.out.println(s2.count + " " + s3.count);
    }
}
```

Çıktı (Output)

```
Toplam: 3
99 99
```

- **`count`** bellekte **tek** bir yerde tutulur; tüm object'ler onu görür.
- **`s1.count = 99`** yazıldığında **`s2`** ve **`s3`** de 99 görür.
- Okunabilirlik için **`Student.count`** biçimi tercih edilmelidir.
- Tipik kullanım: sayaçlar, sabitler (**`static final`**).

Local vs Instance vs Static

Özellik	Local	Instance	Static
Tanımlandığı yer	Method / block içinde	Class içinde, method dışında	Class içinde, static keyword ile
Bellekte oluşma	Method/block çalıştığında	Object oluşturulduğunda (new)	Class yüklendiğinde (bir kez)
Kopya sayısı	Her çağrıda yeni	Her object için bir tane	Tüm class için tek
Default değer	Yok (initialize zorunlu)	Var: 0, false, null	Var: 0, false, null
Erişim	Sadece kendi scope'unda	obj.name	ClassName.name
Ömür (lifetime)	Block bitene kadar	Object erişilebilir kaldıkça	Program/class yüklü kaldıkça

Java'da "global variable" yoktur; birden çok yerden erişilecek veriler static field olarak tanımlanır.

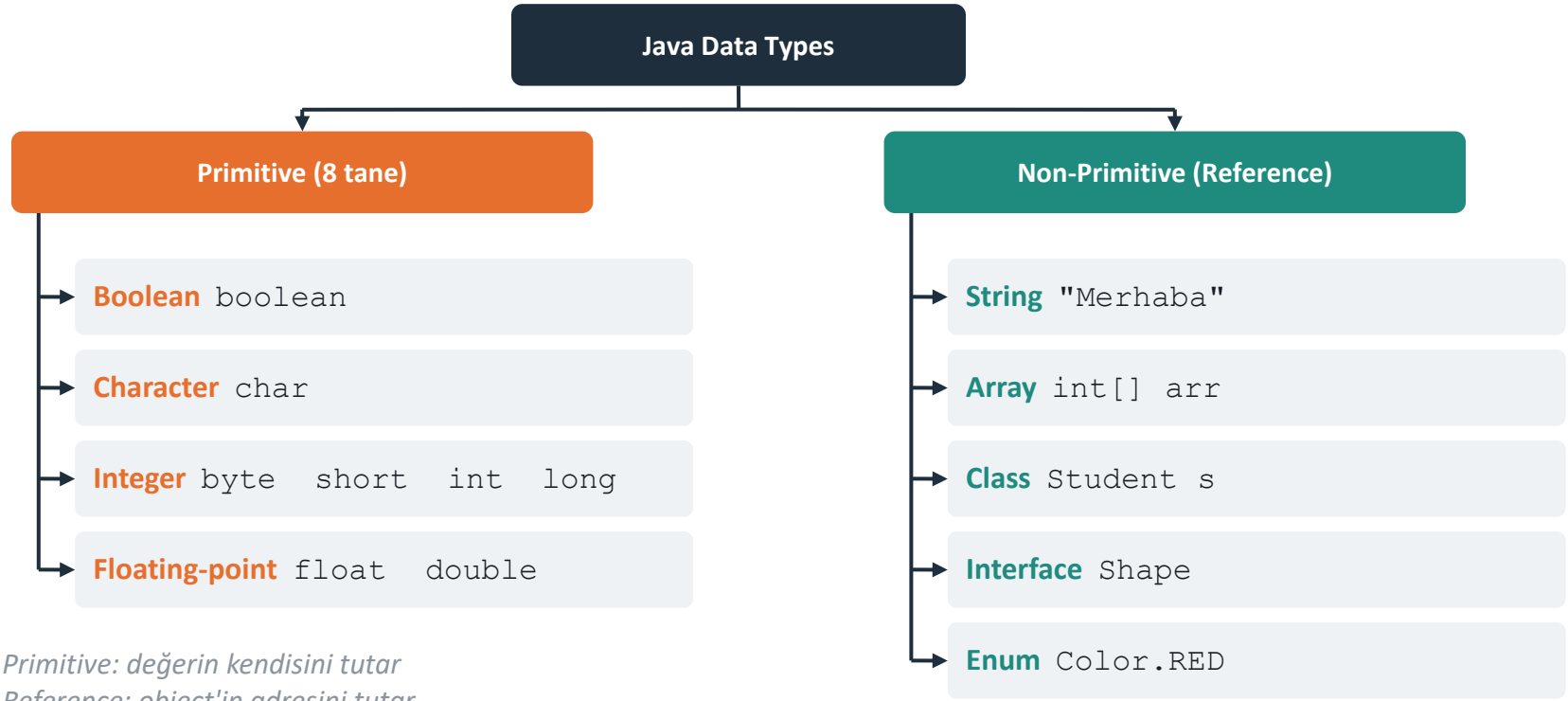
Bölüm 2



Data Types

Primitive ve non-primitive (reference) type'lar

Java Data Types Hiyerarşisi



Primitive: değerin kendisini tutar
Reference: object'in adresini tutar

8 Primitive Data Type

Type	Boyut	Aralık (Range)	Default	Örnek literal
`byte`	8 bit	-128 … 127	0	`(byte) 100`
`short`	16 bit	-32,768 … 32,767	0	`30000`
`int`	32 bit	-2,147,483,648 … 2,147,483,647	0	`42, -7, 0xFF`
`long`	64 bit	≈ ±9.22 × 10^18	0L	`15000000000L`
`float`	32 bit	≈ ±3.4 × 10^38 (~6-7 anlamlı basamak)	0.0f	`3.14f`
`double`	64 bit	≈ ±1.8 × 10^308 (~15-16 anlamlı basamak)	0.0	`3.14, 2e3`
`char`	16 bit	'\u0000' … '\uffff' (0 … 65,535)	'\u0000'	`'A', '\u0041'`
`boolean`	JVM'e bağlı*	true / false	false	`true`

** Java Language Specification boolean için boyut tanımlamaz; çoğu JVM pratikte 1 byte kullanır. Default değerler yalnızca field'lar (instance/static) ve array elemanları için geçerlidir; local variable'lar default değer almaz.*

Integer Type'lar ve Literal'lar

```java
byte b = 100;
short s = 30_000;  // _ : okunabilirlik (Java 7+)
int population = 85_000_000;
long distance = 15_000_000_000L;  // L eki şart
int hex = 0xFF;  // hexadecimal
int bin = 0b1010;  // binary

System.out.println(b + " " + s);
System.out.println(population + " " + distance);
System.out.println(hex + " " + bin);

int max = Integer.MAX_VALUE;
System.out.println(max);
System.out.println(max + 1);  // overflow!
System.out.println(Long.MAX_VALUE);
```

Çıktı (Output)

```
100 30000
85000000 15000000000
255 10
2147483647
-2147483648
9223372036854775807
```

- `L` eki olmadan `15_000_000_000` int sayılır → "integer number too large" hatası.
- **Overflow** hata üretmez; değer sessizce başa sarar (wrap-around).
- Çok büyük tamsayılar için `long` veya `BigInteger`.

Floating-Point Type'lar: float ve double

```java
float f = 234.5f;  // f eki zorunlu
double d = 12.3;  // varsayılan ondalık tip
double sci = 1.5e3;  // scientific notation

System.out.println(f + " " + d + " " + sci);
System.out.println(10 / 4);  // int / int
System.out.println(10 / 4.0);  // double sonuç
System.out.println(1.0f / 3);  // float hassasiyeti
System.out.println(1.0 / 3);  // double hassasiyeti
System.out.println(0.1 + 0.2);  // sürpriz!
System.out.println(0.1 + 0.2 == 0.3);
```

Çıktı (Output)

```
234.5 12.3 1500.0
2
2.5
0.33333334
0.3333333333333333
0.30000000000000004
false
```

- Ondalık literal'lar varsayılan olarak **`double`**'dır: **`float f = 3.5;`** → compile error.
- IEEE 754 binary formatı 0.1 gibi sayıları **tam** temsil edemez.
- Para hesaplarında **`BigDecimal`** kullanın; ondalıkları **`==`** ile karşılaştırmayın.

char ve boolean

```java
char c = 'A';
char u = '\u0041';  // Unicode: yine 'A'
char tr = 'ş';  // Türkçe karakter de olur
System.out.println(c + " " + u + " " + tr);

System.out.println(c + 1);  // int'e dönüşür
System.out.println((char) (c + 1)); // 'B'
c++;  // char artırılabilir
System.out.println(c);
System.out.println((int) 'a');  // Unicode değeri

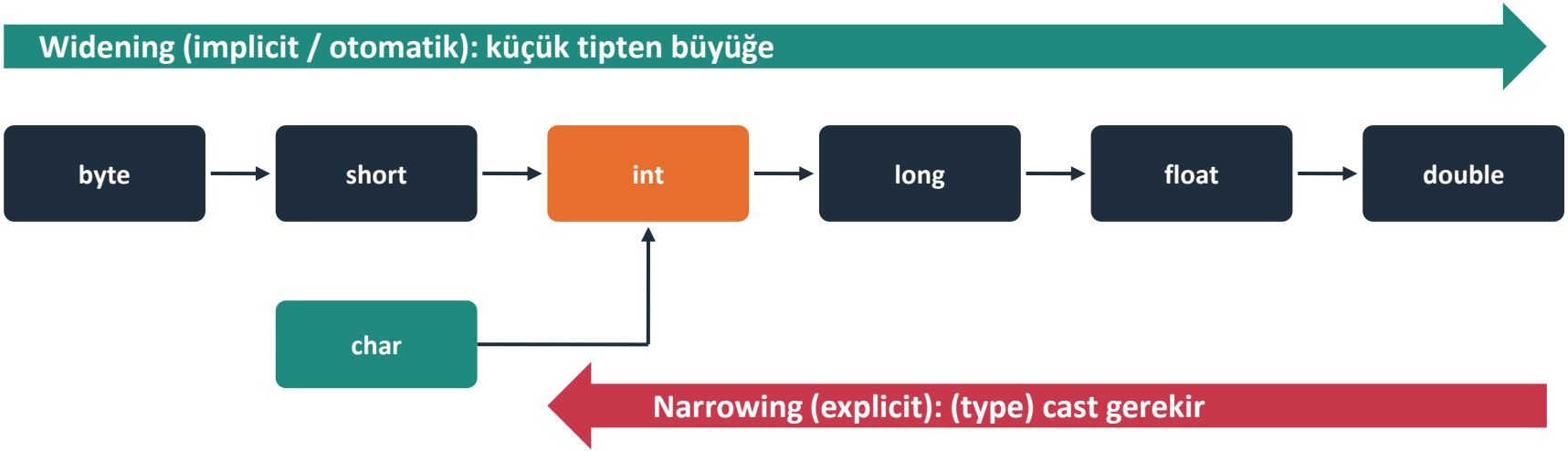
boolean isPassed = 75 >= 50;
boolean isEmpty = false;
System.out.println(isPassed + " " + !isEmpty);
```

Çıktı (Output)

```
A A ş
66
B
B
97
true true
```

- **`char`** 16-bit **Unicode** karakter tutar (ASCII değil); bu yüzden 2 byte'tır.
- Aritmetik işlemde **`char`** önce **`int`**'e yükseltilir.
- **`boolean`** sadece **`true`**/**`false`** alır; C'deki gibi `0`/`1` kullanılamaz: **`boolean b = 1;`** → HATA.

Type Casting: Widening ve Narrowing



- **Widening**: veri kaybı yoktur; derleyici otomatik yapar: `double d = 10;` (yalnız `long`→`float`/`double`'da hassasiyet kaybı olabilir).
- **Narrowing**: kesirli kısım atılır, aralık dışı değer **taşar**: `int i = (int) 9.99;` → 9.
- `boolean` hiçbir tipe cast edilemez.

Type Casting Örnekleri

```java
int a = 10;
double d = a;                    // widening
System.out.println(d);

double price = 10.99;
int p = (int) price;             // narrowing: kesilir
System.out.println(p);
System.out.println(Math.round(price));  // yuvarlama

int big = 130;
byte b = (byte) big;             // aralık dışı!
System.out.println(b);

byte x = 10, y = 20;
// byte z = x + y;               // HATA: x + y -> int
byte z = (byte) (x + y);
System.out.println(z);
```

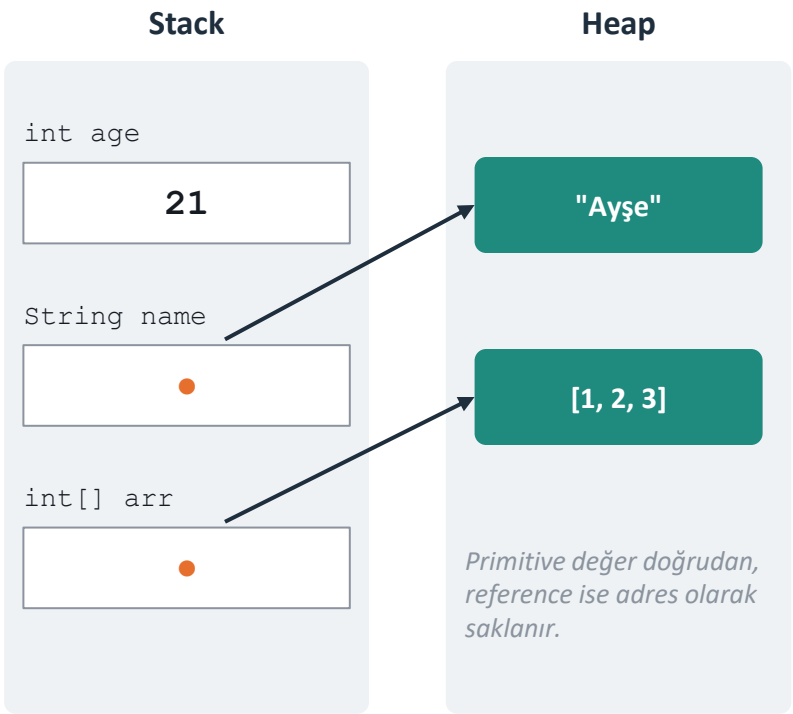
Çıktı (Output)

```
10.0
10
11
-126
30
```

- `(int)` kesirli kısmı **atar** (truncate), yuvarlamaz; yuvarlama için `Math.round()`.
- byte aralığı -128…127: 130 sığmaz → 130 − 256 = **−126** (two's complement).
- `byte`, `short`, `char` aritmetikte önce `int`'e yükseltilir (**numeric promotion**).

Non-Primitive (Reference) Data Types

- Değerin kendisini değil, object'in bellekteki **adresini (reference)** tutar.
- Class, interface, array, enum ve **`String`** bu gruptadır; class ve interface'leri programcı tanımlayabilir.
- Default değeri **`null`**'dır; **`null`** üzerinde method çağırmak → **`NullPointerException`**.
- Method'ları vardır: **`name.length()`**, **`name.toUpperCase()`**.
- İsimleri büyük harfle başlar: **`String`**, **`Scanner`**, **`Integer`**.



Non-Primitive Type'lar ile Çalışmak

```java
enum Color { RED, GREEN, BLUE }

public class Main {
    public static void main(String[] args) {
        String str = "Merhaba";
        int[] arr = {1, 2, 3, 4, 5};
        Color clr = Color.BLUE;

        System.out.println(str.length());
        System.out.println(str.toUpperCase());
        System.out.println("Boyut: " + arr.length);
        System.out.println(clr);

        String s2 = null;
        System.out.println(s2 == null);
    }
}
```

Çıktı (Output)

```
7
MERHABA
Boyut: 5
BLUE
true
```

- **`String` immutable**'dır: **`toUpperCase()`** yeni bir String döndürür, **`str`** değişmez.
- Array'in boyu **`length`** field'ı ile, String'in boyu **`length()`** method'u ile alınır.
- **`enum`**: sabit değerler kümesi (günler, renkler, durumlar).

Wrapper Class'lar ve Autoboxing

```java
Integer x = 42;  // autoboxing: int -> Integer
int y = x;  // unboxing: Integer -> int

int n = Integer.parseInt("123");  // String -> int
double r = Double.parseDouble("2.5");
String s = String.valueOf(3.5);  // double -> String

System.out.println(n + 1);
System.out.println(r * 2);
System.out.println(s + 1);
System.out.println(Integer.MAX_VALUE);
System.out.println(Character.isDigit('7'));
System.out.println(Integer.toBinaryString(y));
```

Çıktı (Output)

```
124
5.0
3.51
2147483647
true
101010
```

- Her primitive'in bir **wrapper class**'ı vardır: **`int`→`Integer`**, **`char`→`Character`**; diğerleri baş harfi büyük yazılır (**`Double`**, **`Boolean`**…).
- Collection'lar (**`ArrayList<Integer>`**) primitive tutamaz; wrapper gerekir.

Bölüm 3



Operators

Variable'lar ve değerler üzerinde işlem yapmak

Java Operator Kategorileri

Unary

`+ - ++ -- ! ~`
Tek operand alır

Arithmetic

`+ - * / %`
Temel matematik

Relational

`< > <= >= == !=`
boolean döndürür

Logical

`&& || !`
Koşulları birleştirir

Bitwise

`& | ^ ~`
Bit düzeyinde işlem

Shift

`<< >> >>>`
Bitleri kaydırır

Assignment

`= += -= *= /= %=`
Değer atar

Ternary

`c ? a : b`
Tek satırlık if-else

Arithmetic Operators

```java
int a = 10, b = 3;
System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);      // integer division
System.out.println(a % b);      // modulus (kalan)
System.out.println(-10 % 3);    // işaret soldan gelir
System.out.println(10.0 / 3);
System.out.println(7.5 % 2);    // double'da da çalışır
System.out.println(10.0 / 0);
// System.out.println(10 / 0); // ArithmeticException!
```

Çıktı (Output)

```
13
7
30
3
1
-1
3.3333333333333335
1.5
Infinity
```

- **`int / int`** → kesirli kısım atılır.
- Çift/tek kontrolü: **`n % 2 == 0`**
- **`10 / 0`** → **ArithmeticException**; **`10.0 / 0`** → **`Infinity`**

Unary Operators: ++ ve --

```java
int x = 10;
System.out.println(+x);
System.out.println(-x);
System.out.println(x++);  // önce kullan, sonra artır
System.out.println(x);
System.out.println(++x);  // önce artır, sonra kullan
System.out.println(x--);  // önce kullan, sonra azalt
System.out.println(--x);  // önce azalt, sonra kullan

boolean ok = true;
System.out.println(!ok);  // logical complement
System.out.println(~5);  // bitwise complement
```

Çıktı (Output)

```
10
-10
10
11
12
12
10
false
-6
```

- **Post-increment `x++`**: ifadede eski değer kullanılır.
- **Pre-increment `++x`**: önce artırılır, yeni değer kullanılır.
- **`~x`** her biti ters çevirir: **`~5 = -6`**

Relational ve Logical Operators

```java
int a = 10, b = 20, c = 5;
System.out.println(a < b);
System.out.println(a == b);
System.out.println(a != b);
System.out.println(a < b && b > c);
System.out.println(a > b || a > c);
System.out.println(!(a < b));

// Short-circuit: (a < c) false -> ++a çalışmaz
if (a < c && ++a > 0) { }
System.out.println("a = " + a);
// & short-circuit yapmaz: iki taraf da çalışır
if (a < c & ++a > 0) { }
System.out.println("a = " + a);
```

Çıktı (Output)

```
true
false
true
true
true
false
a = 10
a = 11
```

- Relational operator'ler her zaman **boolean** döndürür.
- **`&&`**: ilk koşul false ise ikinciye bakmaz. **`||`**: ilk koşul true ise ikinciye bakmaz.
- **`null`** kontrolü **`&&`** operatörünün soluna yazılmalı.

Logical Operators: Truth Table

A	B	A && B	A || B	!A	A ^ B (XOR)
`true`	`true`	`true`	`true`	`false`	`false`
`true`	`false`	`false`	`true`	`false`	`true`
`false`	`true`	`false`	`true`	`true`	`true`
`false`	`false`	`false`	`false`	`true`	`false`

Kural: `&&` iki taraf da true ise true; `||` en az biri true ise true; `^` iki taraf farklı ise true.

Dikkat: `!` **logical NOT**'tur (boolean ters çevirir); `~` ise **bitwise complement**'tir (bitleri ters çevirir).

Bitwise Operators: Bit Bit Hesaplama

OR (|)

```
  1010    (10)
| 0101    (5)
------
  1111    (15)
```

AND (&)

```
  1011    (11)
& 0101    (5)
------
  0001    (1)
```

XOR (^)

```
  1011    (11)
^ 0101    (5)
------
  1110    (14)
```

NOT (~)

```
~ 0…0101   (5)
----------
  1…1010   (-6)
~x = -x - 1
```

Kullanım: izin/flag bitleri, `(n & 1) == 0` ile çift sayı kontrolü, maskeleme, hash ve şifreleme algoritmaları. `&`, `|`, `^` boolean'larla da çalışır ama short-circuit yapmaz.

Shift Operators

```java
System.out.println(10 << 2);     // 10 * 2^2
System.out.println(15 << 4);     // 15 * 2^4
System.out.println(20 >> 2);     // 20 / 2^2
System.out.println(20 >> 3);     // 20 / 2^3 (aşağı)
System.out.println(-20 >> 2);    // işaret korunur
System.out.println(-20 >>> 2);   // soldan 0 gelir

System.out.println(Integer.toBinaryString(20));
System.out.println(Integer.toBinaryString(20 >> 2));
```

Çıktı (Output)

```
40
240
5
2
-5
1073741819
10100
101
```

- **`x << n`** ≈ $x \times 2^n$; **`x >> n`** ≈ $x \div 2^n$ (aşağı yuvarlar).
- **`>>` signed**: soldan işaret biti kopyalanır.
- **`>>>` unsigned**: soldan 0 gelir; negatif sayı büyük bir pozitife dönüşür.

Assignment Operators

```java
int a = 10;
a += 3;   System.out.println(a);   // a = a + 3
a -= 4;   System.out.println(a);
a *= 2;   System.out.println(a);
a /= 4;   System.out.println(a);
a %= 3;   System.out.println(a);

short s = 10;
s += 5;           // OK: implicit cast içerir
// s = s + 5;     // HATA: s + 5 -> int
s = (short) (s + 5);
System.out.println(s);

int x, y, z;
x = y = z = 7;    // sağdan sola (right-to-left)
System.out.println(x + y + z);
```

Çıktı (Output)

```
13
9
18
4
1
20
21
```

- `a op= b` aslında `a = (T)(a op b)` demektir; **gizli bir cast** içerir.
- Assignment **sağdan sola** çalışır.
- İki tarafın aynı tip olması gerekmez; widening otomatik yapılır: `double d = 5;` geçerlidir.

Ternary Operator (? :)

```java
int a = 2, b = 5;
int min = (a < b) ? a : b;
System.out.println("min: " + min);

int x = 10, y = 15, z = 13;
int max = (x > y) ? (x > z ? x : z)
                  : (y > z ? y : z);
System.out.println("max: " + max);

int number = 13;
String type = (number % 2 == 0) ? "çift" : "tek";
System.out.println(number + " sayısı " + type);

int score = 72;
System.out.println(score >= 50 ? "Geçti" : "Kaldı");
```

Çıktı (Output)

```
min: 2
max: 15
13 sayısı tek
Geçti
```

- Söz dizimi: **`condition ? trueValue : falseValue`**
- Java'daki tek **üç operand'lı** operator'dür; bir **değer üretir** (expression).
- İç içe ternary okunabilirliği düşürür; karmaşık koşullarda if-else tercih edin.

Operator Precedence (Öncelik Sırası)

Öncelik	Operator'ler	Yön
1 Postfix	`expr++  expr--`	→
2 Unary	`++expr --expr +  -  ~  !`	←
3 Multiplicative	`*  /  %`	→
4 Additive	`+  -`	→
5 Shift	`<<  >>  >>>`	→
6 Relational	`<  >  <=  >=  instanceof`	→
7 Equality	`==  !=`	→
8 Bitwise	`&  ^  |  (bu sırayla)`	→
9 Logical	`&&  ||  (bu sırayla)`	→
10 Ternary	`? :`	←
11 Assignment	`=  +=  -=  *=  /=  ...`	←

Örnek

`10*10/5+3-1*4/2`

1) * ve / soldan sağa:

`100/5 → 20`

`1*4 → 4, 4/2 → 2`

2) + ve - soldan sağa:

`20 + 3 - 2`

= 21

Emin değilseniz parantez kullanın: okunabilirlik artar.

Bölüm 4



Control Statements

Programın akışını yönetmek

Control Flow Statement Türleri

Varsayılan akış yukarıdan aşağıya sıralıdır (sequential). Control statement'lar bu sırayı koşula göre değiştirir veya tekrarlar.



Decision Making

- `if` statement
- `if-else`
- `if-else-if` ladder
- nested `if`
- `switch` statement



Loop Statements

- `for` loop
- for-each (enhanced for)
- `while` loop
- `do-while` loop



Jump Statements

- `break`
- `continue`
- `return`

İlk Bakış: Decision ve Loop Birlikte

```java
int sum = 0;
for (int j = 1; j <= 10; j++) {  // init; condition; update
    sum = sum + j;
}
System.out.println("1..10 toplamı: " + sum);

if (sum > 50) {
    System.out.println("Toplam 50'den büyük");
} else {
    System.out.println("Toplam 50 veya daha küçük");
}

int i = 0;
while (i <= 10) {  // 0'dan 10'a çift sayılar
    System.out.print(i + " ");
    i += 2;
}
```

Çıktı (Output)

```
1..10 toplamı: 55
Toplam 50'den büyük
0 2 4 6 8 10
```

- **`for`** başlığındaki üç kısım **noktalı virgül** ile ayrılır (kaynaktaki virgüllü syntax hatalıdır).
- **`print`** satır atlamaz, **`println`** atlar.

String Karşılaştırma: == mi equals() mi?

✗ == reference'ı karşılaştırır

```
String a = "Java";
String b = new String("Java");
String c = "Java";
System.out.println(a == b);
System.out.println(a == c);
```

Çıktı (Output)

```
false
true
```

- `a == c` true çünkü literal'lar **String pool**'dan gelir; bu tesadüftür.
- Kullanıcı girdisinde (Scanner) == yanlış sonuç verir.

✓ equals() içeriği karşılaştırır

```
String a = "Java";
String b = new String("Java");
String c = "JAVA";
System.out.println(a.equals(b));
System.out.println(a.equalsIgnoreCase(c));
```

Çıktı (Output)

```
true
true
```

- String içeriği her zaman `equals()` ile karşılaştırılır.
- Büyük/küçük harf duyarsız: `equalsIgnoreCase()`.

Sık Yapılan Hatalar

Initialize edilmemiş local variable

`int t; println(t);` → compile error. Local variable'lar default değer almaz.

Literal eklerini unutmak

`float f = 3.5;` → HATA, doğrusu `3.5f`. Büyük long değerlerde `L` eki gerekir.

Integer division

`5 / 2` → 2. Ortalama için: `(double) sum / n`.

Sessiz overflow

`Integer.MAX_VALUE + 1` negatif olur, hata vermez. Gerekirse `long` kullanın.

String'leri == ile karşılaştırmak

Her zaman `equals()` kullanın.

= ile == karışıklığı

`if (x = 5)` Java'da compile error verir; karşılaştırma `==` ile yapılır.

Alıştırma: Çıktı Nedir?

1

Soru 1

```
System.out.println("Toplam: " + 2 + 3);
System.out.println(2 + 3 + " Toplam");
```

2

Soru 2

```
int i = 5;
int j = i++ + ++i;
System.out.println(i + " " + j);
```

3

Soru 3

```
System.out.println(7 / 2 * 2.0);
System.out.println(7 / 2.0 * 2);
```

4

Soru 4

```
byte b = (byte) 200;
char c = (char) ('a' + 2);
System.out.println(b + " " + c);
```

Özet

Variables
Bellekteki alana isim; local (default yok), instance (object başına), static (tek kopya).

Data Types
8 primitive (byte, short, int, long, float, double, char, boolean) ve reference type'lar; widening otomatik, narrowing cast ister.

Operators
Integer division, ++/-- sırası, short-circuit && / ||, compound assignment'taki gizli cast'e dikkat.

Sıradaki ders
Decision making (if-else, switch) ve for / for-each loop'ları.

Kaynak: tpointtech.com Java eğitim sayfaları (içerik genişletilmiş, örnekler eklenmiş ve hatalar düzeltilmiştir).